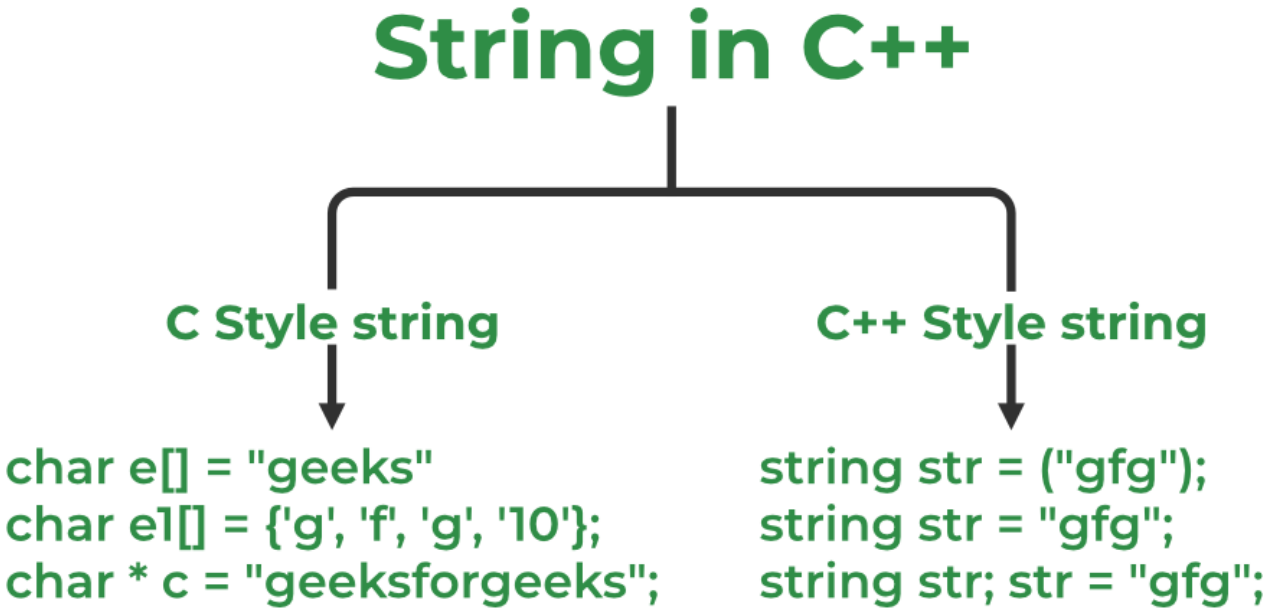
Strings in C++

C++ strings are sequences of characters stored in a char array. Strings are used to store words and text. They are also used to store data, such as numbers and other types of information. Strings in C++ can be defined either using the **std::string class** or the **C-style character arrays.**



. C Style Strings

These strings are stored as the plain old array of characters terminated by a **null character '\0'**. They are the type of strings that C++ inherited from C language.

Syntax:

```
char str[] = "GeeksforGeeks";
```

C++

```cpp
// C++ Program to demonstrate strings
#include <iostream>
using namespace std;

int main()
{

    char s[] = "GeeksforGeeks";
    cout << s << endl;
    return 0;
}
```

2. std::string Class

These are the new types of strings that are introduced in C++ as std::string class defined inside <string> header file. This provides many advantages over conventional C-style strings such as dynamic size, member functions, etc.

Syntax:

std::string str("GeeksforGeeks");

C++

```cpp
// C++ program to create std::string objects
#include <iostream>
using namespace std;

int main()
{

    string str("GeeksforGeeks");
    cout << str;
    return 0;
}
```

Syntax:

```
std::string str(number,character);
```

Example:

C++

```
#include <iostream>
using namespace std;

int main()
{
    string str(5, 'g');
    cout << str;
    return 0;
}
```

Ways to Define a String in C++

Strings can be defined in several ways in C++. Strings can be accessed from the standard library using the string class. Character arrays can also be used to define strings. String provides a rich set of features, such as searching and manipulating, which are commonly used methods. Despite being less advanced than the string class, this method is still widely used, as it is more efficient and easier to use. Ways to define a string in C++ are:

- **Using String keyword**
- **Using C-style strings**

1. Using string Keyword

It is more convenient to define a string with the string keyword instead of using the array keyword because it is easy to write and understand.

Syntax:

string s = "GeeksforGeeks";

string s("GeeksforGeeks");

C++

```cpp
// C++ Program to demonstrate use of string keyword
#include <iostream>
using namespace std;

int main()
{

    string s = "GeeksforGeeks";
    string str("GeeksforGeeks");

    cout << "s = " << s << endl;
    cout << "str = " << str << endl;

    return 0;
}
```

2. Using C-style strings

Using C-style string libraries functions such as strcpy(), strcmp(), and strcat() to define strings. This method is more complex and not as widely used as the other two, but it can be useful when dealing with legacy code or when you need performance.

char s[] = {'g', 'f', 'g', '\0'};

char s[4] = {'g', 'f', 'g', '\0'};

char s[4] = "gfg";

char s[] = "gfg";

C++

```cpp
// C++ Program to demonstrate C-style string declaration
#include <iostream>
using namespace std;

int main()
{

    char s1[] = { 'g', 'f', 'g', '\0' };
    char s2[4] = { 'g', 'f', 'g', '\0' };
    char s3[4] = "gfg";
    char s4[] = "gfg";

    cout << "s1 = " << s1 << endl;
    cout << "s2 = " << s2 << endl;
    cout << "s3 = " << s3 << endl;
    cout << "s4 = " << s4 << endl;

    return 0;
}
```

Another example of C-style string:

C++

```cpp
#include <iostream>
using namespace std;

int main()
{
    string S = "Geeeks for Geeks";
    cout << "Your string is= ";
    cout << S << endl;

    return 0;
}
```

How to Take String Input in C++

String input means accepting a string from a user. In C++. We have different types of taking input from the user which depend on the string. The most common way is to take input with **cin** keyword with the extraction operator (>>) in C++. Methods to take a string as input are:

- **cin**
- **getline**
- **stringstream**

1. Using Cin

The simplest way to take string input is to use the **cin** command along with the stream extraction operator (>>).

Syntax:

cin>>s;

C++

```cpp
// C++ Program to demonstrate string input using cin
#include <iostream>
using namespace std;

int main() {

      string s;

    cout<<"Enter String"<<endl;
      cin>>s;

      cout<<"String is: "<<s<<endl;
    return 0;
}
```

2. Using getline

The getline() function in C++ is used to read a string from an input stream. It is declared in the <string> header file.

Syntax:

getline(cin,s);

Example:

C++

```cpp
// C++ Program to demonstrate use of getline function
#include <iostream>
using namespace std;

int main()
{

    string s;
    cout << "Enter String" << endl;
    getline(cin, s);
    cout << "String is: " << s << endl;
    return 0;
}
```

3. Using stringstream

The stringstream class in C++ is used to take multiple strings as input at once.

Syntax:

stringstream stringstream_object(string_name);

C++

```cpp
// C++ Program to demonstrate use of stringstream object
#include <iostream>
#include <sstream>
#include<string>

using namespace std;

int main()
{

    string s = " GeeksforGeeks to the Moon ";
    stringstream obj(s);
    // string to store words individually
    string temp;
    // >> operator will read from the stringstream object
    while (obj >> temp) {
        cout << temp << endl;
    }
    return 0;
}
```

How to Pass Strings to Functions?

In the same way that we pass an array to a function, strings in C++ can be passed to functions as character arrays. Here is an example program:

Example:

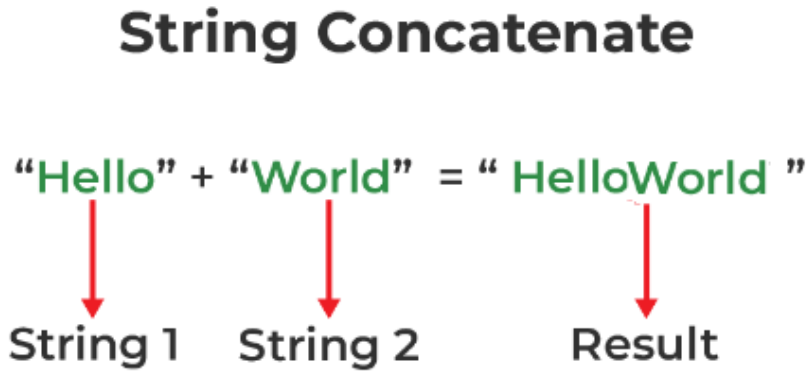
C++

```cpp
// C++ Program to print string using function
#include <iostream>
using namespace std;

void print_string(string s)
{
    cout << "Passed String is: " << s << endl;
    return;
}

int main()
{

    string s = "GeeksforGeeks";
    print_string(s);

    return 0;
}
```

String Concatenation in C++

Read | Courses | Practice | Video

The string is a type of data structure used for storing characters. Concatenating strings in C++ is one of the most discussed topics related to strings. There are multiple methods to concat strings using user-defined methods, and a couple of methods for the concatenation of strings using pre-defined methods. Let's check on all of these methods.



Methods of Concatenate String

There are 6 methods to Concatenate String as mentioned below:

1. Using append( ) Function.
2. Using '+' Operator.
3. Using strcat( ) Function.
4. Using C++ for Loop.
5. Using Inheritance.
6. Using the Friend Function and strcat() Function.

1. Using append() Function

The append() function is a member function of the **std::string class**. Using this function, we can concatenate two std::string objects (C++ style strings) as shown in the below example.

Below is the C++ program for string concatenation using the append() function:

C++

```cpp
// C++ Program for string
// concatenation using append
#include <iostream>
using namespace std;

// Driver code
int main()
{
    string init("this is init");
    string add(" added now");

    // Appending the string.
    init.append(add);

    cout << init << endl;
    return 0;
}
```

2. Using '+' Operator

This is the easiest method for the concatenation of two strings. The + operator **adds strings** and returns a concatenated string. This method only works for C++ style strings (std::string objects) and doesn't work on C style strings (character array).

Syntax:

```
string new_string = init + add;
```

Below is the C++ program for string concatenation using '+' operator:

C++

```
// C++ Program for string
// concatenation using '+' operator
#include <iostream>
using namespace std;

// Driver code
int main()
{
    string init("this is init");
    string add(" added now");

    // Appending the string.
    init = init + add;

    cout << init << endl;
    return 0;
}
```

3. Using strcat( ) Function

The C++ strcat( ) function is a built-in function defined in **<string.h>** header file. This function concatenates the two strings **init** and **add** and the result is stored in the **init** string. This function only works for C-style strings (character arrays) and doesn't work for C++-style strings (std::string objects).

Syntax:

```
char * strcat(char * init, const char * add);
```

Below is the C++ program for string concatenation using strcat() function:

Below is the C++ program for string concatenation using strcat() function:

C++

```cpp
// C++ Program for string
// concatenation using strcat
#include <iostream>
#include <string.h>
using namespace std;

// Driver code
int main()
{
    char init[] = "this is init";
    char add[] = " added now";

    // Concatenating the string.
    strcat(init, add);

    cout << init << endl;

    return 0;
}
```

4. Using for Loop

Using a loop is one of the most basic methods of string concatenation. Here, we are adding elements one by one while traversing the whole string and then another string. The final result will be the concatenated string formed from both strings.

Below is the C++ program for string concatenation using for loop:

Below is the C++ program for string concatenation using for loop:

C++

```cpp
// C++ Program for string
// concatenation using for loop
#include <iostream>
using namespace std;

// Driver code
int main()
{
    string init("this is init");
    string add(" added now");

    string output;

    // Adding element inside output
    // from  init
    for (int i = 0; init[i] != '\0'; i++)
    {
        output += init[i];
    }

    // Adding element inside output
    // fromt add
    for (int i = 0; add[i] != '\0'; i++)
    {
        output += add[i];
    }

    cout << output << endl;
    return 0;
```